Documentation for CAN Driver Development

Nazmus Sakib Graduate Student, Aerospace and Ocean Engineering, Virginia Tech, Blacksburg, VA, USA. nazmus.sakib@vt.edu

Binay Rijal Undergraduate Student, Computer Science, Virginia Tech, Blacksburg, VA, USA. binaypr@vt.edu

Acronyms

MAVLink micro air vehicle communication protocol. 6
ROS robot operating system. 2

ADU air-data unit. 6

CAN controller area network. 1–4, 6–10, 12

ESC electronic speed controller. 6, 9–12

GPS global positioning system. 3, 6
GPU graphical processing unit. 6
GUI graphical user interface. 6

HID human interface device. 4

I2C inter-integrated circuit. 4, 6
IDE integrated development environment. 4

OOP object-oriented programming. 2

PWM pulse width modulation. 4, 6, 10, 11

RPM rotations per minute. 9–11

SPI serial peripheral interface. 4

UART universal asynchronous receiver/transmitter. 4, 6
UAV unmanned aerial vehicle. 2, 3, 5, 6
USB universal serial bus. 4, 6

1 Introduction

The objective of this document is to capture the several steps required to develop a fully functional custom controller area network (CAN)-bus driver for a `PX4` autopilot, using CAN supported microcontrollers, for acquiring data. The document contains detailed step-by-step instructions on how to develop such drivers starting from scratch. This document is expected to be a guide to the users new to the `PX4`

environment. Before a driver can be developed and tested, the user needs to be familiar with the following topics:

- object-oriented programming (OOP) (specifically C++)
- flight control software (specifically PX4)
- different protocols for the transmitting and receiving data (specifically CAN)
- microcontroller development system (specifically the Teensy board)

1.1 Basics of object-oriented programming (OOP)

The object-oriented programming (OOP) is a programming paradigm that organizes code into reusable and self-contained objects. Each object is an instance of a class, and classes define the properties (attributes) and behaviors (methods) that objects of that class possess. OOP is based on the principles of encapsulation, inheritance, and polymorphism, providing a way to structure and design software in a modular and maintainable manner.

Here are the key concepts in OOP that are required to understand and develop C++ codes:

1. **Classes and Objects:** A class is a blueprint or template and an object is an instance of that class.
2. **Encapsulation:** It is the bundling of data (attributes) and the methods (functions) that operate on that data into a single unit, i.e., a class.
3. **Inheritance:** It is a mechanism that allows a class (subclass or derived class) to inherit properties and behaviors from another class (superclass or base class).
4. **Polymorphism:** It provides a way to use a single interface to represent different types of objects.
5. **Constructor and Destructor:** Constructors are special methods used to initialize an object when it is created. Destructors are used to clean up resources when an object is no longer in use.

OOP is widely used in modern programming languages like Java, C++, Python, etc. However, it is sufficient to know only C++ as that is the language used for CAN driver development in PX4.

1.2 Introduction to CAN

The controller area network (CAN) is a protocol originally intended for usage in automotive applications and is used to communicate information across a set of "Electronic Stability Control" units, "Engine Control" units, and various associated sensors. Unlike frameworks such as robot operating system (ROS), it does not require a master/slave connection, thus allowing for the "democratization" of message transfer. As unmanned aerial vehicle (UAV) systems developed, the potential for the utilization of the CAN protocol in unmanned systems was quickly recognized and led to the development of the UAVCAN implementation – and associated implementations like CANaerospace and CANopen.

Here are the key concepts in CAN that are essential for developing CAN applications:

1. **CAN Messages:** Information is communicated in CAN through "frames", consisting of a "header" field, a "data" field, and a "tail" field.
2. **Frame Structure:** CAN frames have a defined structure with specific fields for organizing data.
3. **Message Limitations:** Standard CAN frames traditionally have an 8-byte data field limit, influencing data transmission in certain applications.
4. **Extensions:** Implementations like UAVCAN, CANaerospace, and CANopen extend CAN capabilities for specific use cases.

CAN provides several advantages that make it useful in various applications:

1. **Decentralized Communication:** Unlike master/slave systems, CAN allows for decentralized message transfer, enhancing flexibility.

2. **Wide Adoption:** Widely used in automotive and industrial applications, CAN has become a standard for reliable communication.
3. **Protocol Extensions:** Specialized implementations like `UAVCAN` cater to the unique needs of unmanned systems.
4. **Defined Structure:** The structured frame format ensures organized and efficient data transmission.

Flight control software like `PX4` supports two CAN communication protocols:

- `DroneCAN` is the primary CAN protocol recommended by the flight control software. It is well supported by `PX4`, is a mature product with extensive peripheral support, and has had years of testing.
- `Cyphal` is a much newer protocol that allows more flexibility and configuration, especially on larger and more complex vehicles. It has not yet seen significant adoption and `PX4` support for `Cyphal` is a "work in progress".
- Other CAN software protocols for drones such as `KDECAN` are not yet supported by any of the flight control software.

`DroneCAN`, because of its extensive support within the flight control software projects like `PX4`, is chosen as the communication protocol in this document. Section 1.4 talks about how to get started on `DroneCAN` implementation.

1.3 What is a PX4?

`PX4` autopilot is an open-source flight control software suite designed for unmanned aerial vehicles and other robotic vehicles. It provides a comprehensive set of firmware, middleware, and ground control software to enable the control, navigation, and communication of autonomous vehicles. `PX4` is widely used in the drone industry and is known for its flexibility, modularity, and support for a variety of hardware platforms. The key features of `PX4` include:

1. **Flight Control Firmware:** `PX4` provides flight control firmware that runs on autopilot hardware. This firmware is responsible for stabilizing the vehicle, executing control commands, and managing sensors and actuators.
2. **Modular Architecture:** `PX4` has a modular architecture that allows users to select and customize components based on their specific requirements. This modularity facilitates the integration of different sensors, communication modules, and other peripherals.
3. **Hardware Support:** `PX4` supports a wide range of hardware platforms, including popular flight controllers like `Pixhawk`, `Pixhawk 2`, `Pixracer`, and others. This flexibility allows users to choose hardware that suits their application needs.
4. **Sensors Integration:** `PX4` supports integration with various sensors, including accelerometers, gyroscopes, magnetometers, barometers, global positioning system (GPS) modules, and more. Additionally, custom sensors can be integrated into the `PX4` with proper drivers for collecting and storing data which is the primary reason for developing this documentation.
5. **Control Modes:** `PX4` supports different flight modes, including "stabilized mode", "altitude hold", "position hold", and "autonomous missions". Users can switch between modes to adapt to different flight scenarios.
6. **Mission Planning:** `PX4` includes tools for mission planning and waypoint navigation. Users can define waypoints, set flight paths, and plan complex missions using ground control stations compatible with `PX4`.
7. **Communication Protocols:** It supports communication protocols such as `MAVLink`, which is a lightweight communication protocol for UAVs. `MAVLink` facilitates communication between the autopilot and ground control stations.

The open-source nature and active community of PX4 make it a popular choice for developers and organizations working on autonomous vehicle projects. An alternative to PX4 is the Ardupilot flight control software suite.

1.4 Driver Development on a Teensy Board

A Teensy board refers to a family of microcontroller development boards designed and manufactured by Paul Stoffregen's Teensy Projects (PJRC). These boards are compact, versatile, and feature-rich, making them popular choices for embedded systems and electronics projects. Teensy boards are known for their small form factor, ease of use, and compatibility with the Arduino development environment. Some of the common Teensy boards include the Teensy 3.x and the Teensy 4.x series. Key features of the Teensy boards include:

1. **Arduino Compatibility:** Teensy boards are designed to be compatible with the Arduino development environment, making it easy for users familiar with Arduino to start working with the Teensy.
2. **USB Support:** Teensy boards often have built-in universal serial bus (USB) support, allowing them to act as USB devices or hosts. This feature is useful for various applications, including USB communication and human interface device (HID) emulation.
3. **Digital and Analog I/O:** Teensy boards provide a range of digital and analog input/output pins, enabling interfacing with sensors, actuators, and other electronic components. This is one of the reasons for using the Teensy board.
4. **Communication Interfaces:** Teensy boards typically support various communication interfaces such as inter-integrated circuit (I2C), serial peripheral interface (SPI), universal asynchronous receiver/transmitter (UART), and controller area network (CAN), providing flexibility for connecting to other devices and peripherals. Having support for CAN is the primary reason for using the Teensy board in the work described in this document.
5. **PWM Output:** Teensy boards offer pulse width modulation (PWM) outputs, allowing users to control analog-like signals for applications like motor control or LED brightness.
6. **Community Support:** The Teensy community is active and provides support through forums and documentation. PJRC, the organization behind Teensy, maintains an active presence to assist users and developers.

To develop and run codes on the Teensy boards PlatformIO is used. PlatformIO integrated development environment (IDE) is a professional collaborative platform for embedded development that allows users to quickly develop and deploy software to different platforms and frameworks. It contains hardware architecture, operating system, runtime libraries, and other essential components of several development boards: Atmel AVR, Freescale Kinetis, Infineon XMC, Intel MCS-51 (8051), Teensy, etc. Code development environments like PlatformIO simplify the software development process on the Teensy board easier. Other alternatives include: Arduino IDE with Teensyduino add-on, Visual Micro, and CircuitPython. PlatformIO is chosen as the IDE, in this document, since it is easier to use with VS Code. If VS Code is not being used, then the other alternative environments can be used. The PlatformIO IDE can compile, run, and deploy the code on the board making the entire development process simple, easy, and efficient. It is very simple to add PlatformIO via VS Code by going into its "extension" toolbar and searching for PlatformIO. The PlatformIO IDE then gives the users access to several libraries related to the different supported boards. For example, it contains the FlexCAN_t4 and arduino libraries which are needed for the driver development on a Teensy board. If for some reason VS Code is not used, then the required libraries have to be manually downloaded and configured so that the code development is possible.

As mentioned in Section 1.2, DroneCAN is chosen as the protocol because of its extensive PX4 support. However, the original implementation of DroneCAN was intended for utilization in fully

Linux-based systems. This means that many of the built-in library functions, variables, and message definitions defined in the `DroneCAN` driver would not work with the `Arduino`-based `Teensy` hardware. `FlexCAN_t4` is a library that provides the low-level communication drivers to interact with the `Teensy` hardware. So, there is the `DroneCAN` library supported by `PX4` which contains libraries suited for Linux-based systems, and the `FlexCAN` libraries which is used to interact with the `Teensy` hardware. To make these two different libraries compatible, developers at Bolder Flight Systems created modified versions of `DroneCan` and `FlexCAN` libraries. These modified libraries provide the necessary modified functions, parameters, variables, and classes to implement `DroneCAN` on the `Teensy` board. This solves the mismatch of different message definitions and between the Linux-based codes and the `Arduino`-based `Teensy` hardware. The custom libraries developed by Bolder Flight Systems, used later in Section 2.2, are:

- https://github.com/bolderflight/dronecan
- https://github.com/bolderflight/FlexCAN
- https://github.com/bolderflight/circle_buf

1.5 Analogy of the Flight Controller Components With Computer Hardware and Software

The different jargon used throughout Section 1 can leave the user somewhat confused. Similarities between the different hardware/software components of a UAV flight controller with that of an actual computer can help users understand the different roles played by each of the components; see Fig. 1.

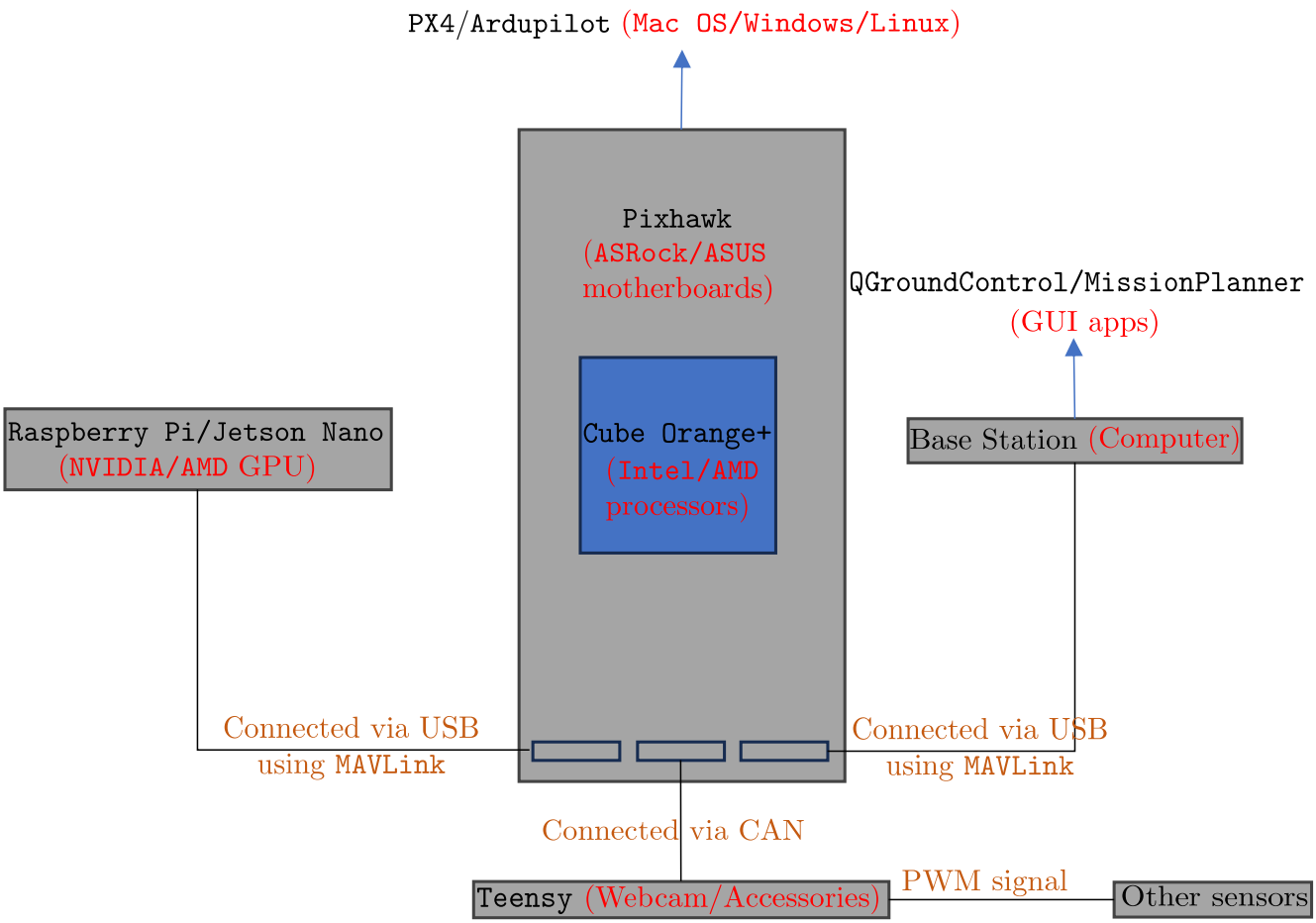


Fig. 1 Different UAV autopilot components and their computer hardware/software counterparts.

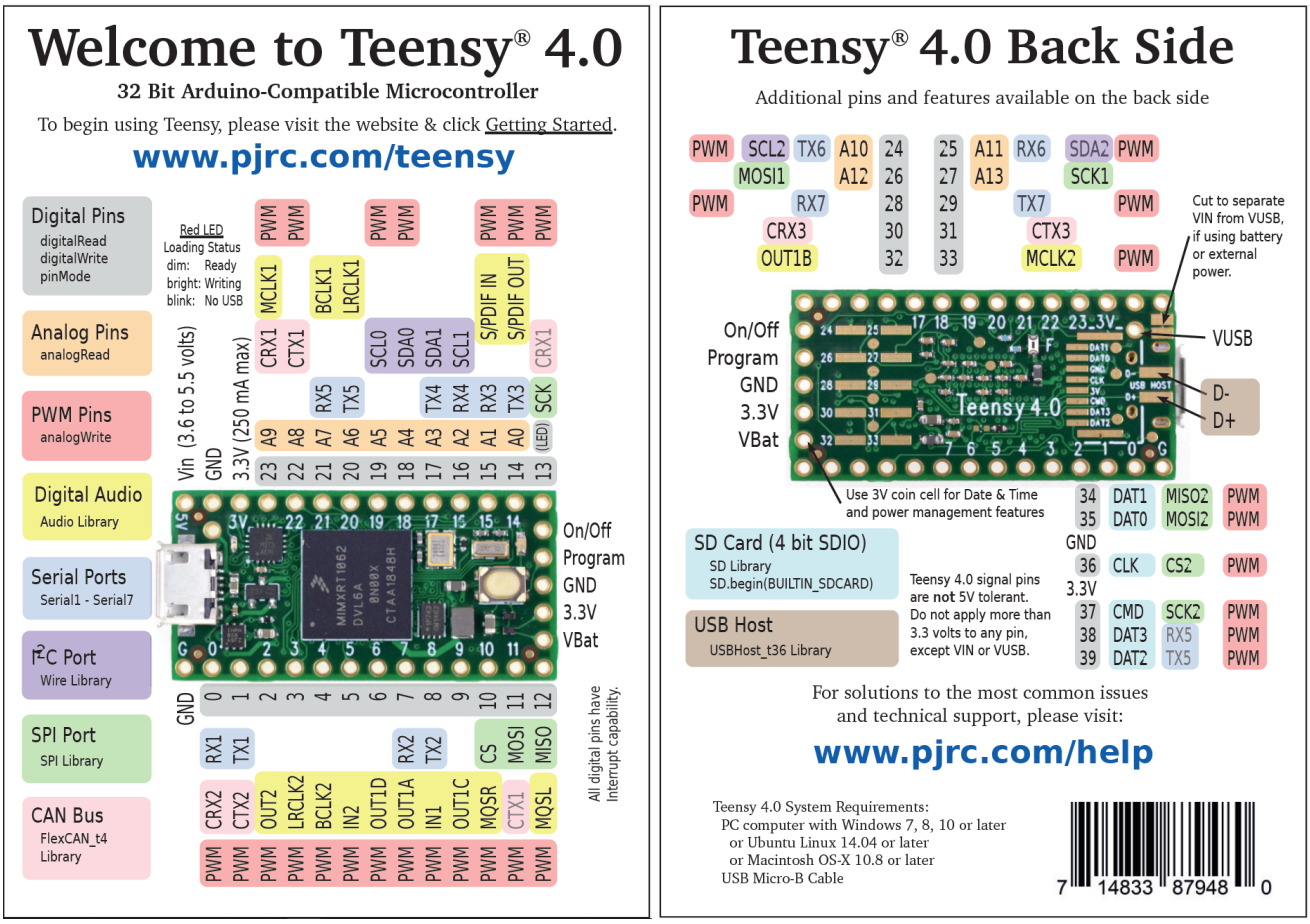
- **`Pixhawk` board:** It refers to a family of open-source flight control hardware boards used in UAVs and other robotic vehicles that follow the "`Pixhawk` standard" which are a set of specifications and

features to define the design and functionality of ubiquitous autonomous systems. All the Pixhawk boards have specifically designed ports, channels, pins, and data communication protocols that are used for communicating with a UAV. As such it is similar to the work that a "motherboard/main board" does in a computer. Companies like CubePilot and Holybro develop their carrier boards following the Pixhawk standard similar to the motherboard manufacturers like ASUS, ASRock, etc.

- **Standard flight control module:** These modules include a microcontroller, sensors (such as accelerometers, gyroscopes, magnetometers, barometers), and other components necessary for UAV control. These can be thought of as the processors for the UAV similar to the actual processors in a computer. Companies like CubePilot and Holybro develop their modules, like the Cube Orange, similar to the processor chip developers like Intel and AMD.
- **Autopilot software:** PX4 and Ardupilot autopilots are both open-source flight control software suites designed for UAVs and other robotic vehicles. They interact with the flight control module and provide vehicle stability, navigation, and control commands based on the sensor readings. Their work is similar to the operating software in a computer like Mac OS, Windows, and Linux.
- **Companion computer:** A companion computer is like a partner to the flight controller (e.g., ArduPilot or PX4) in an autonomous vehicle. It handles more complex tasks that improve the vehicle's overall capabilities. Usually, the flight control module is a tiny computer that takes in raw sensor data, figures out the vehicle's states and issues commands to servos and motors to achieve a desired objective like "staying in place" or "moving to a specific location". However, these modules are quite limited when it comes to their computing power, memory, and storage. In contrast, a companion computer is usually much more powerful. The companion computer gets data from the flight controller, often along with information from other attached sensors, over USB through a protocol named micro air vehicle communication protocol (MAVLink). The usage of a companion computer in a UAV is similar to how a graphical processing unit (GPU) is used in a computer. Although having a GPU is not necessary for running a computer, it does provide additional computational capabilities that increase the efficiency of a computer. There are different brands of companion computers available in the market like the Raspberry Pi manufactured by the Raspberry Pi foundation and the Jetson series manufactured by NVIDIA similar to the GPU manufactures like Intel, AMD, and NVIDIA.
- **Interactive GUIs:** Applications like QGroundControl and MissionPlanner are run on a "base station" that lets users interact with the vehicle thorough a graphical user interface (GUI). The GUI makes the interaction process easier and simpler. It displays sensor data, radio channel configuration, data analysis toolset, sensor calibrator, and other commands for manipulating UAVs. These applications are similar to any software or application existing on a computer. For example, apps like Windows Task Manager or Open Hardware Monitor can display information about the usage and temperature of different processors, memories, GPUs, and storage.
- **Custom sensors:** The Teensy board is an interfacing device that establishes connection between the Pixhawk board and a custom sensor. The standard Pixhawk board and module together with the PX4 firmware supports sensors necessary for flying/controlling a robotic vehicle like accelerometer, magnetometer, gyroscope, GPS receiver, etc. Any additional sensor, which are not strictly necessary for flight but provides useful data for research or other purposes, needs to be manually integrated to the flight controller firmware which is where the Teensy board comes into play. The Teensy receives PWM signals from a custom sensor (example, air-data unit (ADU), motor's electronic speed controller (ESC), sonic anemometer, etc.) and converts the data into useful messages understood and stored by the PX4 software. The communication between Teensy and PX4 can happen using I2C, UART, CAN, etc. In this document, the connection utilizing CAN is focused on. Since these "custom sensors" are not strictly necessary for manipulating an autonomous robot, they can be considered as accessories. For example, a computer user might require a camera to perform various tasks. Typically the camera system interfaces with the computer via USB and the user is required to install software to access the camera. Any custom

sensor is similar to a computer camera where the `Teensy` board interfaces with the sensor using different `C++` scripts and then it relays the information to the `PX4` operating software using the CAN protocol.

This document focuses on how to develop these `C++` scripts (also known as "drivers") for custom sensors and how to configure the scripts to output proper messages that `PX4` can understand. Additionally, this document also talks about the hardware setup and the wiring diagram necessary for enabling data transfer.



(a) (b)

Fig. 2 (a) Front of a `Teensy 4.0` board. (b) Back of a `Teensy 4.0` board.

2 Step-by-step Guide

The full data transmission architecture from the sensor to the `PX4` can be described by this data flow diagram: custom sensor –> `Teensy` board –> CAN transceiver –> `PX4`. Here, the custom sensor generates the data of interest. The `Teensy` board receives this data onto its CAN bus driver and modifies and structures the data for `PX4` to understand and store. For example, unit conversion of the data can take place on the `Teensy` board to offload some computational burden from the `PX4`. CAN operates using a differential signaling scheme, where two wires `CANH` (CAN high) and `CANL` (CAN low) carry complementary signals. The CAN transceiver converts the digital signals from the CAN controller into the differential voltage levels required for transmission on the bus which is then read by the `PX4` software and stored in the `Pixhawk` board. Figure 2 gives the front and back of the `Teensy 4.0` boards used in this driver development which contains useful information about the board. By inspection, `Pins 0/1`, `13/11`, `23/22`, and `30/31` are the CAN buses that can receive/transmit signals using the `FlexCAN_t4` library. `PX4`, on the other hand, requires the signal to be in the `CANH` and `CANL` format. The digital

conversion of the CAN data happens in the CAN transceiver. The `CRX1/CTX1` pin on the `Teensy` board connects to the digital R_{XD}/T_{XD} pin on the CAN transceiver which is then converted to appropriate `CANH` and `CANL` signals for the `PX4` to read and store; see Fig. 3.

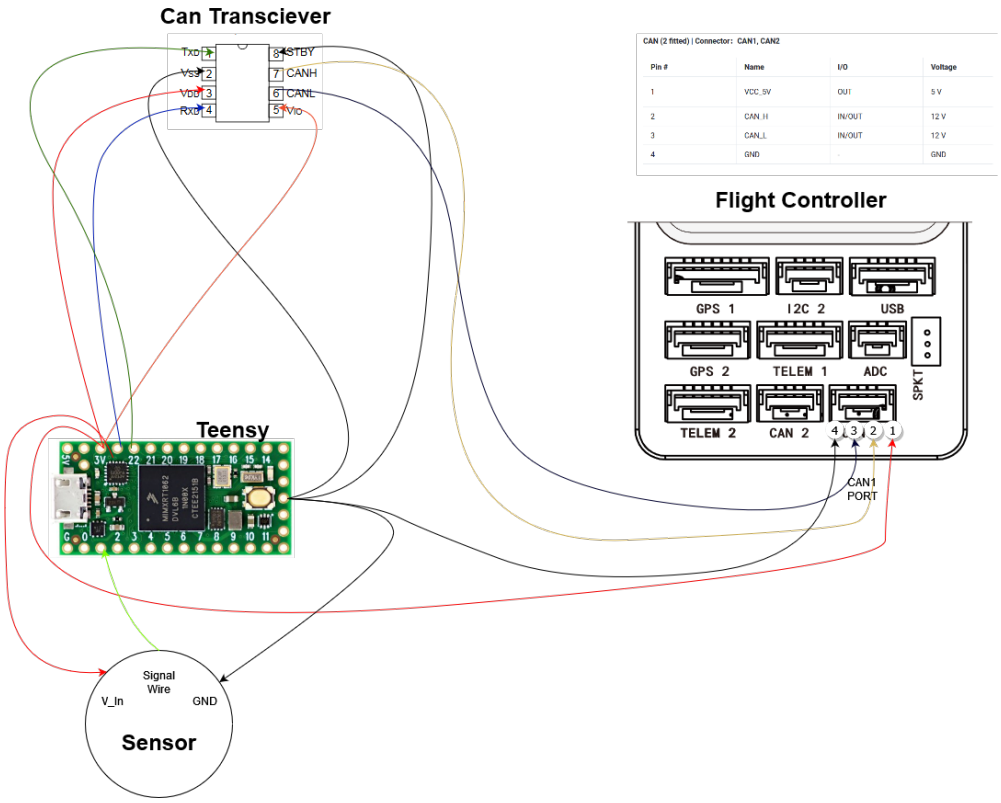


Fig. 3 Wiring Diagram

2.1 Driver development in the PX4

The driver development on the `PX4` side consists of reading the CAN-bus data coming in from the `Teensy` board and then storing the data in appropriate data types.

1 Open the Linux `terminal` and navigate to the desired directory where the `PX4` code should be installed/cloned. Use the command: `$git clone https://github.com/PX4/PX4-Autopilot.git --recurse-submodules`. This command recursively updates the `main` repository and its associated `submodules`. Make sure the `main` `PX4` branch is cloned, which can be found in the top-left corner of the file list; see Fig. 4. After the cloning is complete type on the Linux

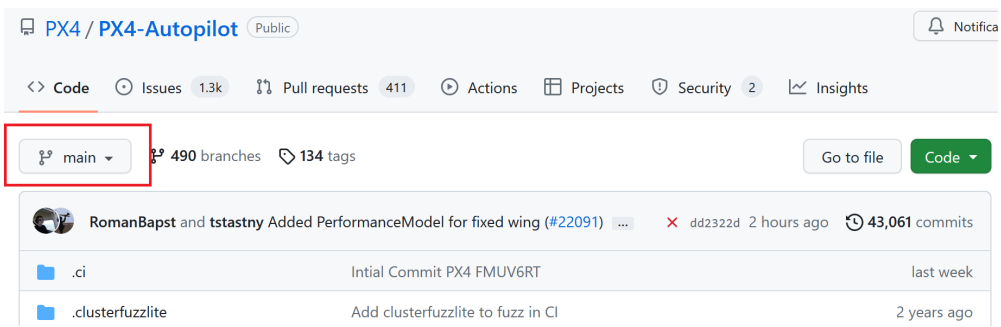


Fig. 4 Location of the `main` branch in `GitHub`.

`terminal` `$git checkout release/1.14` to switch to the most-stable release version (here,

it is version 1.14). Again, update the submodules recursively using `$git submodule update --recursive`. Next, open the entire PX4 directory in any code editor like VS Code.

2 The next few steps assume that the PX4 repository, named PX4-Autopilot-v1.14.CAN, are downloaded to a location ~/can/PX4-Autopilot-v1.14.CAN. Navigate to the directory: /.../PX4-Autopilot-v1.14.CAN/src/drivers/uavcan/sensors/ and copy one of the existing .cpp and .hpp files and rename it to the desired driver. For this example, the airspeed.cpp and the airspeed.hpp files were copied and renamed to custom_rpm.cpp and custom_rpm.hpp.

3 First, go inside the created .hpp file and make changes as described in the following lines. The changes can also be seen by comparing the original file (airspeed.hpp) with the modified example file (custom_rpm.hpp). Changes to be made:

3.1 Change the name of the overall C++ class. For this example, in line#51, class UavcanAirspeedBridge{} is renamed to class UavcanCustomRPMBridge{}. Please take a look at the words and their capitalizations, those are PX4 conventions and it is recommended to follow their conventions.

3.2 Now change the callback functions to the desired callbacks. These callback functions will have the desired codes that are needed to read and/or write the required data. In line#56 UavcanAirspeedBridge() is renamed to UavcanCustomRPMBridge() which takes in the same parameters. In line#64, void rpm_sub_cb() is the main callback function where the required codes are added which is similar to void ias_sub_cb() in airspeed.hpp. Only one callback is required in custom_rpm.hpp but many more can be constructed based on requirements. The airspeed.hpp has other callbacks like void tas_sub_cb() and void oat_sub_cb(). The parameter passed into the callback void rpm_sub_cb() needs to be of the right UAVCAN message type. These message types can be found under /.../src/drivers/uavcan/libuavcan/dsdl/uavcan/equipment/. This example demonstrates how to obtain and store the rotations per minute (RPM) data from the ESC. The ESC status message found in /.../equipment/esc/1034.Status.uavcan contains a variable int18 rpm required exactly for this example. So the data type of the parameter of the callback function will be <uavcan::equipment::esc::Status>. To make the .hpp file understand what this data type means it needs to be included in the header as #include <uavcan/equipment/esc/Status.hpp>. Looking at the airspeed.hpp file, it is also seen that the data type for the void tas_sub_cb() callback function is <uavcan::equipment::air_data::IndicatedAirspeed> which means the function is working with the IndicatedAirspeed data. Upon inspecting the different messages located in /.../equipment/air_data/1021.IndicatedAirspeed.uavcan. If no such message type exists in the directory /.../equipment/ to store the desired variables then custom message types are needed to be defined. It is strongly recommended that these custom message types are not created as the message types on the microcontrollers like the Teensy board must also match these custom message types. This means that the repositories developed by the Bolder Flight Systems, used later in Section 2.2, need to be modified which requires advanced knowledge on CAN library and driver development.

3.3 Then define the typedef of the data type. In C++, typedef is used for aliasing existing data types, user-defined data types, and pointers to a more meaningful name. Here the standard uavcan::MethodBinder<> data structure of data type <UavcanCustomRPMBridge* ... <uavcan::equipment::esc::Status> &)> is given a new name RPMCbBinder. Again, the capitalizations follow the standard PX4 convention. The file airspeed.hpp also defines similar typedef for each of its different data types used in its different functions. This new typedef is then used in the uavcan::Subscriber{} class. Create an instance of the uavcan subscriber that subscribes to the data sent to the PX4. The message data type will be the same as defined in the callback function. This is where the connection between the data, received from an external source, and the correspond-

ing PX4 uavcan message takes place; which is why the data types need to match. Next, the data is published/stored to the proper PX4 message topics using uORB. In line#72, uORB::PublicationMulti<esc_status_s> _esc_status_pub{ORB_ID(esc_status)} takes care of the publication. Here, the correct uORB topic should be used as found in ~/can/PX4-Autopilot-v1.14.CAN/msg/ and the topic's header file should be included in the beginning. For example, in custom_rpm.hpp, the uORB topic name used is esc_status which was included in the beginning as #include "ESCStatus.h". Additionally, to avoid compilation errors regarding uORB::PublicationMulti, add #include <uORB/PublicationMulti.hpp> in the beginning. This tells the compiler how to publish custom topics.

4 Moving on to the custom_rpm.cpp file: rename everything from UavcanAirspeedBridge() to UavcanCustomRPMBridge() as defined in the .hpp file. Make sure to include the new header file; that is, replace #include "airspeed.hpp" with #include "custom_rpm.hpp". On line#43 change the NAME field to contain the desired sensor name (for example, "custom_rpm"). Intuitively, this NAME field might be used to indicate where C++ print statements from a code come from in the NuttX shell. Change the C++ string parameter passed into the UavcanSensorBridgeBase to a new name. Here, the name has been changed to "uavcan_custom_rpm". Change the ORB_ID to the correct uORB message being used. It should be the same as the imported/included uORB topic in the header file. In this example, uORB/topic/esc_status.h has been used. After that, modify the name of the callback to the ones defined in the header files. Then use the subscriber initialized in the .hpp file, _sub_rpm_data, to start the callback function also defined in the .hpp file: rpm_sub_cb.

2.2 Driver development in the Teensy

The driver development on the Teensy side consists of reading the PWM signal coming from the motors ESCs, converting them to motor RPM, and publishing to the appropriate CAN data type for PX4 to understand.

1 Create a new project in PlatformIO by selecting the right development board. For example, Teensy 4.0 is the board used here. Let the project be named customRPM. Let its location be in the home directory as: ~/customRPM/

2 After a project has been customized there should be a "lib" folder in the project directory. So, the location would be ~/customRPM/lib/ Clone the following into the lib folder

```
$git clone https://github.com/bolderflight/dronecan
$git clone https://github.com/bolderflight/FlexCAN
$git clone https://github.com/bolderflight/circle_buf
```

These will clone the bolderflight repositories into the lib directory and make the driver development easier.

3 Go to lib/dronecan/example/arduino/pub/pub.ino and copy the entire code into src/main.cpp. This main file will be the file that is run on the Teensy board.

4 On line#36 change <CAN3> to <CANX> bus number that will be used. This is based on the wiring of the board. According to Figs. 2a and 3, the pins 22/23 are connected which correspond to CTX1/CRX1. The last number "1" is the CAN-bus number which is why CAN1 is selected. Use the "case sensitive" search and replace tool in VS Code to ensure that all the ports are appropriately named. On line#37, change the publisher data structure to be the same on the PX4 side: <uavcan::equipment::esc::Status> defined previously in Step 3.3. This tells the Teensy board to publish messages with the correct data type that PX4 understands.

5 Get rid of the CAN transceiver pinMode as it does not seem to affect the data transmission. In line#71 change the publisher to the same data type as mentioned before.

6 In the `setup()` function, add the necessary interrupts to perform the desired tasks to read the desired data above the "Set node mode operational" part. The number of interrupts will depend on the type of task and the total number of sensors being read. These interrupts use callback functions which contain the actual code for performing the desired tasks and these codes vary based on application. The objective of the example drivers is to read the PWM data from the six different motors of a hexacopter multirotor aircraft. The `Teensy` board is thus getting access to six different PWMs coming from the six different motors of the hexacopter. The callback functions then read the length of the PWM signal by measuring the time difference between the "rising" and the "falling" of the pins and convert the data to the RPM value by dividing with the number of motor poles. The "rising" of a pin when a PWM signal is available and the "falling" happens when the signal vanishes. In the code, the `PIN number` is based on the pins where the wiring comes in, that is, the pins where data is being received; see Fig 3.

7 For the callback functions to work properly a minimal delay needs to be added. This should be a very small integer number like 1 ms or 2 ms which tells the board to look for data after "sleeping" for that specified amount of delay. Otherwise, the code becomes unstable.

3 Results

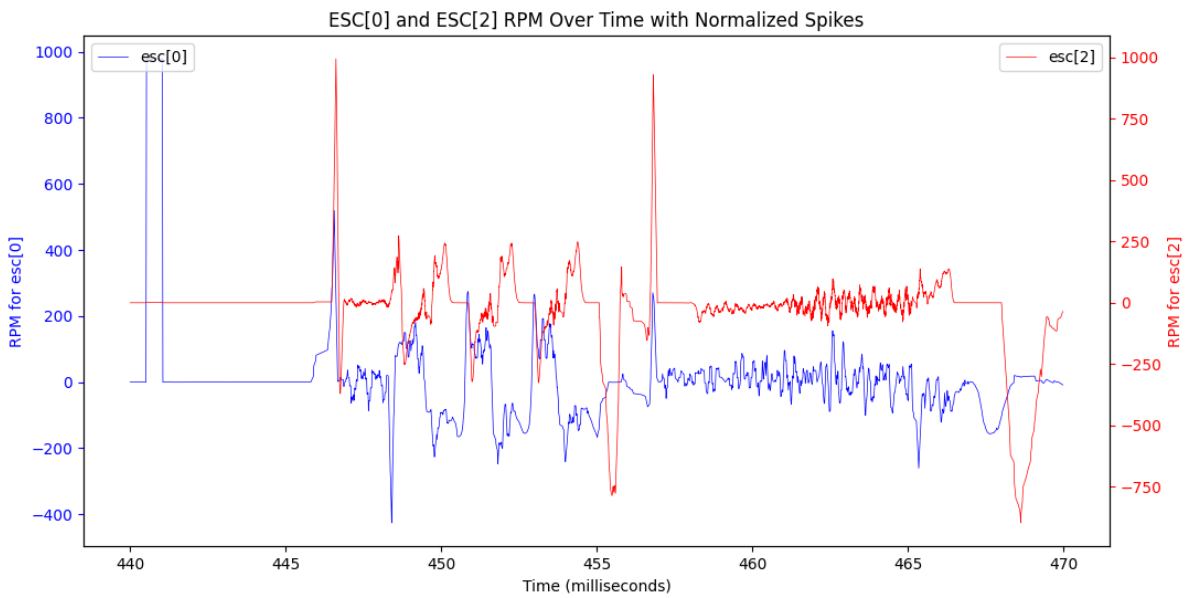


Fig. 5 Normalized ESC values

Figure 5 shows the measured RPM trends of the two ESCs, denoted as `esc[0]` and `esc[2]`. On the graph, the "blue" line corresponds to the RPM values for `esc[0]` – measured directly from a servo tester, while the "red" line depicts the RPM values for `esc[2]` – measured from the "yellow" wire on the hexacopter's ESCs, the x-axis represents time in milliseconds.

An intriguing observation was that the RPM of `esc[0]` was inversely related to `esc[2]`. This implied that as the RPM for `esc[0]` ascended, the RPM for `esc[2]` descended, and vice versa. This occurrence can be seen in Fig. 6. The inverted values corresponded almost directly to the values obtained from the servo tester.

Several factors could contribute to this phenomenon, including the wiring configuration of the ESCs, the settings applied to the ESCs, or the manner in which the motors are linked to the ESCs. Overall, the graph highlights a noteworthy correlation between the RPM of the two ESCs, but the exact cause warrants

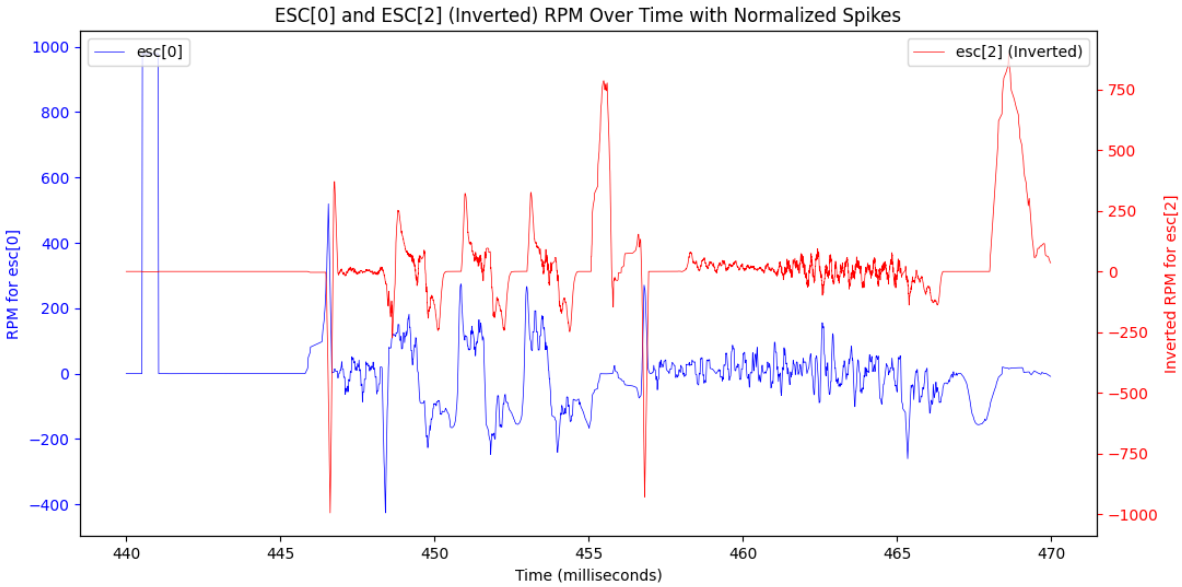


Fig. 6 Normalized ESC values with `esc[2] inverted`

further investigation. The inverted relationship between `esc[0]` and `esc[2]` suggests a potential link and future research could shed light on the underlying factors and their proper calculation. It's worth noting that similar behavior was observed on a different brand of ESCs, having the same CAN-bus connections and driver, suggesting a shared cause.